Practical : 10	Implementation of Kruskal's Algorithm
Date : __ / __ / 2026	

Aim: To implement **Kruskal's algorithm** in C++ to find the **Minimum Spanning Tree (MST)** of a connected, undirected, weighted graph using the **Disjoint Set Union (Union-Find)** data structure.

Objective :- To store a weighted graph as an edge list, build the MST by sorting the edges and rejecting cycle-forming edges, and to analyze and compare its complexity with Prim's algorithm.

A. Graph Representation (Edge List)

1. Start.
2. Read the number of vertices V and the number of edges E.
3. Create an empty list of edges; each edge stores (u, v, w).
4. For each input edge, add (u, v, w) to the list (one entry represents both directions).
5. Display the edge list.
6. Stop.

B. Disjoint Set Union (Union-Find)

1. Start.
2. Create parent[] and rank[]; set parent[i] = i and rank[i] = 0 (every vertex is its own set).
3. find(x): follow parent links up to the root of x, and make every visited vertex point directly to that root (path compression).
4. unite(x, y): find the roots a and b. If a = b, return false (same set, so a cycle).
5. Otherwise attach the root with the smaller rank under the other root, increase the rank if both ranks are equal, and return true.
6. Stop.

C. Kruskal's Algorithm

1. Start.
2. Sort all edges in ascending order of weight.
3. Create a Union-Find structure for V vertices, an empty MST list and total = 0.
4. Take the next edge (u, v, w) from the sorted list.
5. If unite(u, v) returns true, add the edge to the MST and add w to total; otherwise discard it.
6. Repeat steps 4 and 5 until the MST has V - 1 edges or all edges are used.
7. Print the MST edges and the total weight.
8. Stop.

Program A: Kruskal's Algorithm (practical10_kruskal.cpp)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
// Edge of the graph
struct Edge {
    int u, v, w;
};
// Disjoint Set Union (Union-Find) to detect cycles
class DSU {
    vector<int> parent, rnk;
public:
    DSU(int n) {
        parent.resize(n);
        rnk.assign(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    // Find with path compression
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }
    // Union by rank; returns false if x and y are already connected
    bool unite(int x, int y) {
        int a = find(x);
        int b = find(y);
        if (a == b) return false;
        if (rnk[a] < rnk[b]) swap(a, b);
        parent[b] = a;
        if (rnk[a] == rnk[b]) rnk[a]++;
        return true;
    }
};
class Graph {
    int V;
    vector<Edge> edges;
public:
    Graph(int vertices) {
        V = vertices;
    }
    // Add an undirected weighted edge
    void addEdge(int u, int v, int w) {
        edges.push_back({u, v, w});
    }
    // Display the edge list
    void displayGraph() {
        cout << "\n--- Edge List ---" << endl;
        cout << "Edge\tWeight" << endl;
        for (auto &e : edges) {
            cout << e.u << " - " << e.v << "\t" << e.w << endl;
        }
    }
    // Kruskal's Algorithm
    void kruskalMST() {
        // Step 1: sort all edges by weight (ascending)
        vector<Edge> sorted = edges;
        sort(sorted.begin(), sorted.end(),
             [](const Edge &a, const Edge &b) { return a.w < b.w; });
        cout << "\n--- Edges in Sorted Order ---" << endl;
        for (auto &e : sorted) {
            cout << e.u << " - " << e.v << "\t" << e.w << endl;
        }
        // Step 2: pick edges one by one, skip those that form a cycle
        DSU dsu(V);
        vector<Edge> mst;
        int total = 0;
        for (auto &e : sorted) {
```

```
            if (dsu.unite(e.u, e.v)) {
                mst.push_back(e);
                total += e.w;
                if ((int)mst.size() == V - 1) break;
            }
        }
        cout << "\n--- Minimum Spanning Tree (Kruskal's) ---" << endl;
        cout << "Edge\tWeight" << endl;
        for (auto &e : mst) {
            cout << e.u << " - " << e.v << "\t" << e.w << endl;
        }
        if ((int)mst.size() < V - 1) {
            cout << "Graph is not connected, so a full spanning tree is not possible." << endl;
        }
        cout << "Total weight of MST = " << total << endl;
    }
};
int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    cout << "Enter number of edges: ";
    cin >> E;
    Graph g(V);
    cout << "Enter edges (u v weight):" << endl;
    for (int i = 0; i < E; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u < 0 || u >= V || v < 0 || v >= V) {
            cout << "Invalid edge. Vertices must be between 0 and "
                 << V - 1 << "." << endl;
            i--;
            continue;
        }
        g.addEdge(u, v, w);
    }
    g.displayGraph();
    g.kruskalMST();
    return 0;
}
```

Output:

Sample graph used: 6 vertices (0 to 5) and 9 edges (u-v: weight): 0-1: 4, 0-2: 3, 1-2: 1, 1-3: 2, 2-3: 4, 3-4: 2, 4-5: 6, 3-5: 7, 2-4: 5. The same graph is used as in Practical 9, and the MST total weight is again 14.

Output

```
Enter number of vertices: 6
Enter number of edges: 9
Enter edges (u v weight):
0 1 4
0 2 3
1 2 1
1 3 2
2 3 4
3 4 2
4 5 6
3 5 7
2 4 5

--- Edge List ---
Edge    Weight
0 - 1   4
0 - 2   3
1 - 2   1
1 - 3   2
2 - 3   4
3 - 4   2
4 - 5   6
3 - 5   7
2 - 4   5

--- Edges in Sorted Order ---
1 - 2   1
1 - 3   2
3 - 4   2
0 - 2   3
0 - 1   4
2 - 3   4
2 - 4   5
4 - 5   6
3 - 5   7

--- Minimum Spanning Tree (Kruskal's) ---
Edge    Weight
1 - 2   1
1 - 3   2
3 - 4   2
0 - 2   3
4 - 5   6
Total weight of MST = 14


=== Code Execution Successful ===
```

Program B: Performance Analysis (practical10_analysis.cpp)

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
#include <random>
#include <chrono>
using namespace std;
using namespace chrono;
struct Edge {
    int u, v, w;
};
class DSU {
    vector<int> parent, rnk;
public:
    DSU(int n) {
        parent.resize(n);
        rnk.assign(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int x, int y) {
        int a = find(x), b = find(y);
        if (a == b) return false;
        if (rnk[a] < rnk[b]) swap(a, b);
        parent[b] = a;
        if (rnk[a] == rnk[b]) rnk[a]++;
        return true;
    }
};
// Kruskal's Algorithm; returns total weight of the MST
long long kruskalMST(int V, vector<Edge> edges) {
    sort(edges.begin(), edges.end(),
         [](const Edge &a, const Edge &b) { return a.w < b.w; });
    DSU dsu(V);
    long long total = 0;
    int count = 0;
    for (auto &e : edges) {
        if (dsu.unite(e.u, e.v)) {
            total += e.w;
            if (++count == V - 1) break;
        }
    }
    return total;
}
// Build a connected random edge list with V vertices and E edges (fixed seed)
vector<Edge> generateEdges(int V, int E) {
    mt19937 rng(42);
    vector<Edge> edges;
    for (int i = 1; i < V; i++) {                 // spanning chain keeps it connected
        int j = rng() % i;
        edges.push_back({i, j, (int)(rng() % 100 + 1)});
    }
    for (int i = V - 1; i < E; i++) {             // remaining random edges
        int u = rng() % V, v = rng() % V;
        if (u == v) { i--; continue; }
        edges.push_back({u, v, (int)(rng() % 100 + 1)});
    }
    return edges;
}
int main() {
    int sizes[] = {100, 500, 1000, 5000, 10000, 50000, 100000};
    cout << "Kruskal's Algorithm Performance Analysis" << endl << endl;
    for (int V : sizes) {
        int E = 5 * V;
```

```
        vector<Edge> edges = generateEdges(V, E);
        long long weight = 0, us = LLONG_MAX;
        for (int run = 0; run < 3; run++) {          // best of 3 runs
            auto start = high_resolution_clock::now();
            weight = kruskalMST(V, edges);
            auto stop = high_resolution_clock::now();
            us = min(us, (long long)duration_cast<microseconds>(stop - start).count());
        }
        cout << "Number of Vertices = " << V << endl;
        cout << "Number of Edges    = " << E << endl;
        cout << "MST Total Weight   : " << weight << endl;
        cout << "Kruskal's Time     : " << us << " microseconds" << endl;
        cout << "---------------------------------------" << endl;
    }
    return 0;
}
```

Output:

The same random graphs as in Practical 9 were used (V = 100 to 100000, E = 5V, weights 1 to 100). Each time is the best of 3 runs.

```
Kruskal's Algorithm Performance Analysis

Number of Vertices = 100
Number of Edges    = 500
MST Total Weight   : 1217
Kruskal's Time     : 72 microseconds
---------------------------------------
Number of Vertices = 500
Number of Edges    = 2500
MST Total Weight   : 5983
Kruskal's Time     : 400 microseconds
---------------------------------------
Number of Vertices = 1000
Number of Edges    = 5000
MST Total Weight   : 12478
Kruskal's Time     : 909 microseconds
---------------------------------------
Number of Vertices = 5000
Number of Edges    = 25000
MST Total Weight   : 62064
Kruskal's Time     : 5056 microseconds
---------------------------------------
Number of Vertices = 10000
Number of Edges    = 50000
MST Total Weight   : 125388
Kruskal's Time     : 10680 microseconds
---------------------------------------
Number of Vertices = 50000
Number of Edges    = 250000
MST Total Weight   : 624058
Kruskal's Time     : 65251 microseconds
---------------------------------------
Number of Vertices = 100000
Number of Edges    = 500000
MST Total Weight   : 1249614
Kruskal's Time     : 136023 microseconds
---------------------------------------
```

Observation:

When V and E grew 1000 times, the time of Kruskal's algorithm grew from 72 to 136023 microseconds, about 1889 times. For the same change, E log E grows about 2112 times, which matches the theoretical O(E log E) because sorting the edges dominates the running time.

For every size, the MST total weight is exactly the same as the one found by Prim's algorithm in Practical 9, which confirms that both programs are correct. On these sparse graphs Kruskal's algorithm was also faster (136023 against 308077 microseconds for V = 100000).

The exact timings depend on the computer and compiler, so a different machine will show different numbers, but the growth pattern stays the same.

Analysis:

Time Complexity Analysis

Let V be the number of vertices and E the number of edges of the graph.

Kruskal's Algorithm:

Sorting the E edges takes O(E log E).

At most E find/unite operations are performed. With path compression and union by rank, each one takes almost constant amortized time (inverse Ackermann function), so this part is nearly O(E).

Time Complexity = O(E log E) = O(E log V)

Note: since E is at most V^2, log E is at most 2 log V, so O(E log E) and O(E log V) are the same.

Space Complexity Analysis

The edge list and its sorted copy use O(E) space.

The parent[] and rank[] arrays of the Union-Find structure use O(V) space.

Space Complexity = O(V + E)

Comparison of Kruskal's and Prim's Algorithms

Feature	Kruskal's	Prim's
Approach	Adds the cheapest edge of the whole graph that makes no cycle	Grows one tree from a start vertex
Data structure used	Sorted edge list + Union-Find	Min-heap + adjacency list
Graph representation	Edge list	Adjacency list / matrix
Time complexity	O(E log E) = O(E log V)	O(E log V)
Space complexity	O(V + E)	O(V + E)
Best suited for	Sparse graphs	Dense graphs (matrix version $O(V^2)$)
Disconnected graph	Gives a minimum spanning forest	Spans only the start vertex component

Conclusion :-

Kruskal's algorithm builds the Minimum Spanning Tree by sorting all edges by weight and adding them one by one, skipping every edge that would form a cycle. The cycle check is done efficiently by the Union-Find structure:

1. The program stores the graph as an **edge list**, sorts it, and uses **Union-Find** with path compression and union by rank, giving **O(E log E)** time.

2. For the same sample graph as Practical 9, it produces an MST with V - 1 = 5 edges and the same minimum total weight **14**, which matches the result of Prim's algorithm.

3. Kruskal's algorithm works well for sparse graphs, and on a disconnected graph it gives a minimum spanning forest (the program reports that the graph is not connected).